

Voice-Enabled Chatbot using Speech Recognition and Deep Learning

Live: <https://adiag1205-voicebot.static.hf.space> | Source: <https://github.com/karanjain10/voicebot>

1. Objective

A publicly hosted chatbot that takes spoken input, converts it to text, understands it with deep learning models and replies in text and speech. The screen shows the recognized speech, the predicted intent with its probabilities, and the reply.

2. Dataset

A custom intent dataset (`data/intents.json`) written for this project: **21 intents, 233 example sentences** (8 to 17 per intent), each intent with 1 to 3 canned responses. The time and date intents insert the live value at reply time.

Group	Intents
Conversation	greeting, goodbye, thanks, how_are_you, name, creator, capabilities
Utility / fun	joke, time, date, weather, music, motivation, study_tips
AI topics	machine_learning, deep_learning, neural_network, speech_recognition, nlp, artificial_intelligence, ai_comparison

Preprocessing: lowercase, regex word tokenization, unigrams plus bigrams, a vocabulary of 694 features, and a binary bag-of-words vector per sentence. For open-ended questions outside these intents the system uses a pretrained language model (Section 3), so no extra dataset was needed for it.

3. Model architecture

Mic audio	Speech recognition (browser Web Speech API)	Intent MLP (PyTorch)	confidence ≥ 0.55 : intent response	else: Qwen2.5-0.5B answer	Text + spoken reply
-----------	---	----------------------	---	---------------------------------	------------------------

Speech recognition: the browser's SpeechRecognition API (en-US, interim results shown live). It is a pretrained deep-learning recognizer, so no audio handling is needed on our side.

Intent classifier (trained by us): a multi-layer perceptron `Linear(694, 128) - ReLU - Dropout(0.4) - Linear(128, 64) - ReLU - Dropout(0.4) - Linear(64, 21)` with a softmax output. Cross-entropy loss with label smoothing 0.1 (reduces overconfident predictions), Adam optimizer, learning rate 0.003, 300 full-batch epochs. High dropout is used because the dataset is small.

General-knowledge fallback: if the top probability is below 0.55, the utterance goes to **Qwen2.5-0.5B-Instruct**, a pretrained 0.5-billion-parameter transformer (int8 ONNX, about 520 MB) that runs in the browser through `transformers.js` and `WebAssembly`, with a short system prompt and the last three exchanges as context. This makes the bot answer open-ended questions with no server or API key.

Voice output: the reply is spoken with the browser's `speechSynthesis`.

4. Methodology

(1) Wrote the intent dataset. (2) Split it 75/25 per intent (179 train, 54 test, fixed seed) to measure generalization. (3) Trained the MLP and evaluated on the held-out set. (4) Retrained on all 233 sentences for the shipped model. (5) Exported the weights to JSON and re-implemented the forward pass in JavaScript, and checked it gives the same predictions as PyTorch. (6) Deployed as a static page on a Hugging Face Space, which provides the HTTPS the microphone API requires. A Flask + Docker version of the same app is kept in the repository.

5. Results

Measure	Value
Train accuracy (179 sentences)	100%
Held-out test accuracy (54 sentences, 21 classes)	75.9%
Intent answer latency (in browser)	effectively instant (one small forward pass)
General answer latency (Qwen2.5-0.5B, WebAssembly)	about 2.5 to 4 s per short answer

Observations. With only 15 intents the held-out accuracy was 81%; adding six AI-topic intents that share much vocabulary lowered it to 73.5%, and a dedicated comparison intent (for questions such as the difference between machine learning and deep learning) brought it to 75.9%. Label smoothing makes the confidence honest: in-domain queries score about 0.9 and out-of-domain ones such as "what's the capital of France" score below 0.3, which correctly routes them to the language model, which answered "The capital of France is Paris."

6. Limitations and future work

The intent model is bag-of-words and misses paraphrases that share no words with the training set; more data or a pretrained sentence encoder would help. The 0.5B language model is small and can state wrong facts (it answered "why is the sky blue" incorrectly in testing). The first visit downloads about 520 MB, so the site can take up to a minute to become fully ready; built-in topics work immediately. Speech recognition depends on the browser (Chrome, Edge or Safari; not Firefox) and typed input is the fallback. There is no long-term memory beyond the last three exchanges.